

QueueSmart — Smart Queue Management Application

Team: Parham Ebrahimi, Mustafa Ahmed, Kevin Santhosh, Andrew Andari

Application: QueueSmart (front-end only; mock data)

Date: October 3, 2026

1. GitHub Repository Link

The front-end is maintained at:

<https://github.com/Parham-Ebrahimi/QueueSmart>

TAs can validate individual contributions from the commit history on `main`. Authorship is visible on each commit (Parham-Ebrahimi, Mustafa Ahmed / TheRealMufasa, kevin sorts, wynetyme).

2. Design and Development Methodology

Our methodology has **not changed** since Assignment 1. We still work in a short, screen-first cycle:

- Agree on the information architecture and visual system from A1 (teal workspace, sidebar navigation, card-based status).
- Split the required screens among four people so each person owns a coherent slice of the product.
- Implement the front end with mock data, then review navigation, validation, and visual consistency together.

We did not switch process because A2 is the implementation of the A1 design, not a new problem. The only adjustment was practical: later screens (History, Notifications, Queue Management) consume the same mock services as the earlier dashboards, so we added a shared `localStorage` layer instead of each person inventing a separate fake data set. That keeps the UI consistent for Assignments 3 and 4.

Work was divided by role and screen, not by “whoever is free”:

- **Person 1 — Authentication / UI foundation:** login, registration, validation, navigation, shared CSS and components.
- **Person 2 — User interface:** user dashboard, join queue, queue status, mock queue interactions.
- **Person 3 — Admin services:** admin dashboard, service management, create/edit form, open/close queue.
- **Person 4 — Queue administration:** queue management (reorder / remove / serve next), history, notifications, and closing remaining A2 form-spec gaps.

Everyone was responsible for regular GitHub commits, checking navigation between pages, keeping styling consistent, capturing screenshots of their screens, writing contribution notes, and reviewing this document.

3. Front-End Technologies and Responsibilities

QueueSmart is a single-page web app. We chose a small, conventional front-end stack so the A1 layout could be implemented quickly and reused in later assignments.

Technology	Why we used it
React 19	Component-based screens that map one-to-one to the A1 design. Client-side state is enough for mock queues, history, and notifications. No backend is required for A2.
Vite 6	Fast local development and a one-command production build (<code>npm run dev</code> / <code>npm run build</code>).
React Router 7	Real routes for login, registration, user pages, and admin pages. Role-aware navigation is part of the UX, not a single page of tabs.
lucide-react	A consistent icon set that matches the teal / coral A1 palette without pulling in a full UI kit.
Plain CSS	We kept the A1 look (DM Sans + Manrope, 7–8px radii, teal #137c75) by writing CSS directly instead of fighting a component library.
Browser localStorage	Mock persistence for the session, services, active queue, history, and notifications. Acceptable for A2; replaceable with APIs in A3.

How to run the front end:

`npm install` then `npm run dev`, and open `http://127.0.0.1:5173`. Register or sign in with any valid email and a password of at least 8 characters. Choose User or Administrator before submitting

— the two roles have different workspaces.

Who worked on which screens

Group member	GitHub	Screens / features
Parham Ebrahimi	Parham-Ebrahimi	Login, Registration, client-side auth validation, sidebar / top bar, shared components (Brand , FormField , AppShell) and base CSS.
Mustafa Ahmed	TheRealMufasa	User Dashboard, Join Queue, Queue Status, mock join / leave / simulate-next interactions, Account Settings.
Kevin Santhosh	kevinsorts	Admin Dashboard, Service Management (create / edit / delete), open / close queue controls, initial service-form validation.
Andrew Andari	wynetyne	Queue Management (reorder, remove, serve next), History, in-app Notifications, service-form spec fields (description, priority, 100-character name, expected duration), shared persisted mock data.

4. Screenshots of the Front End

Each required screen is labeled below. Additional validation, History, and Notifications captures are included because the assignment also requires those features.

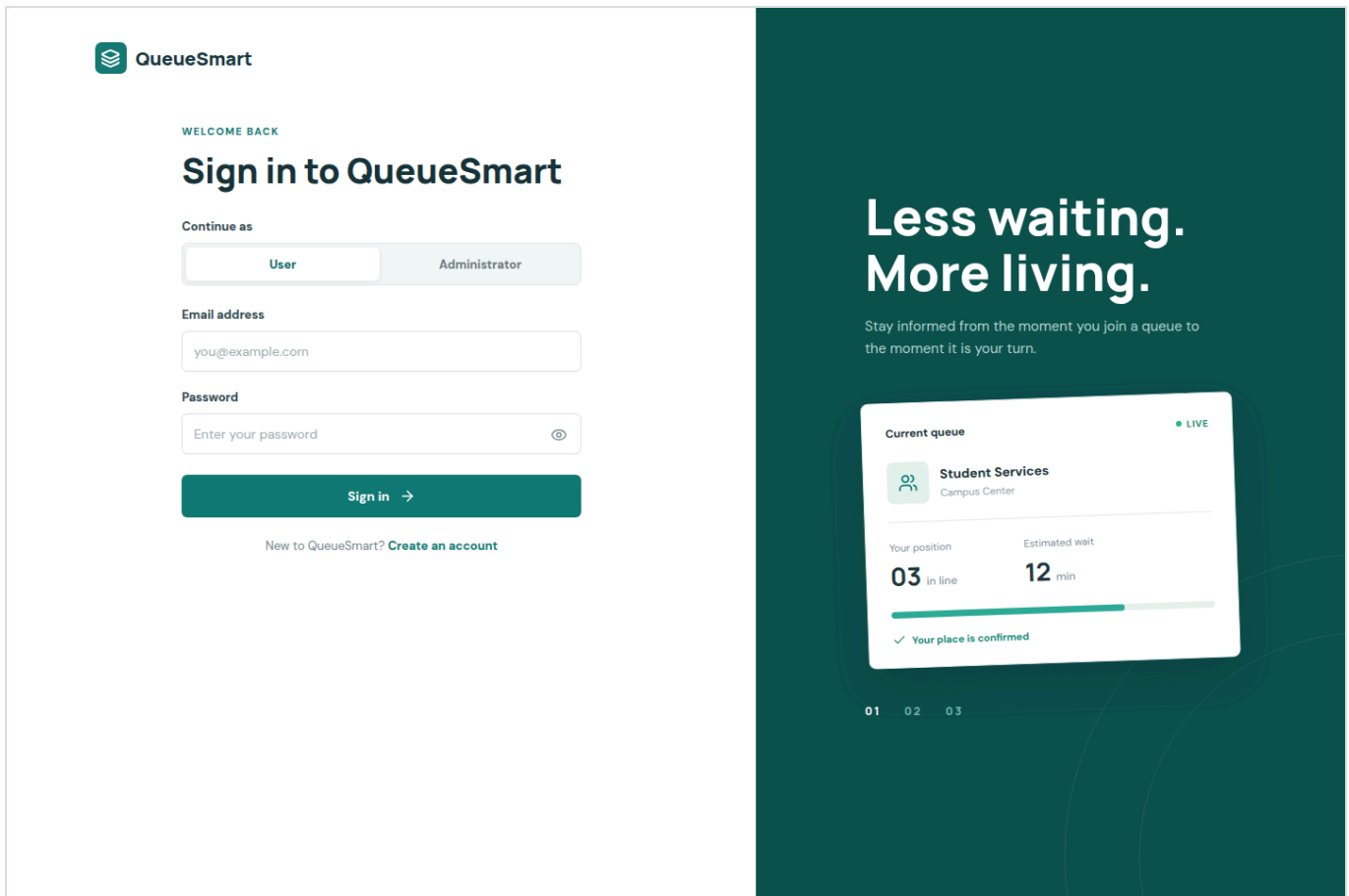


Figure 1. Login. Email (used as username), password, User / Administrator role picker, and a sign-in action. Empty required fields show client-side errors (see Figure 1b).

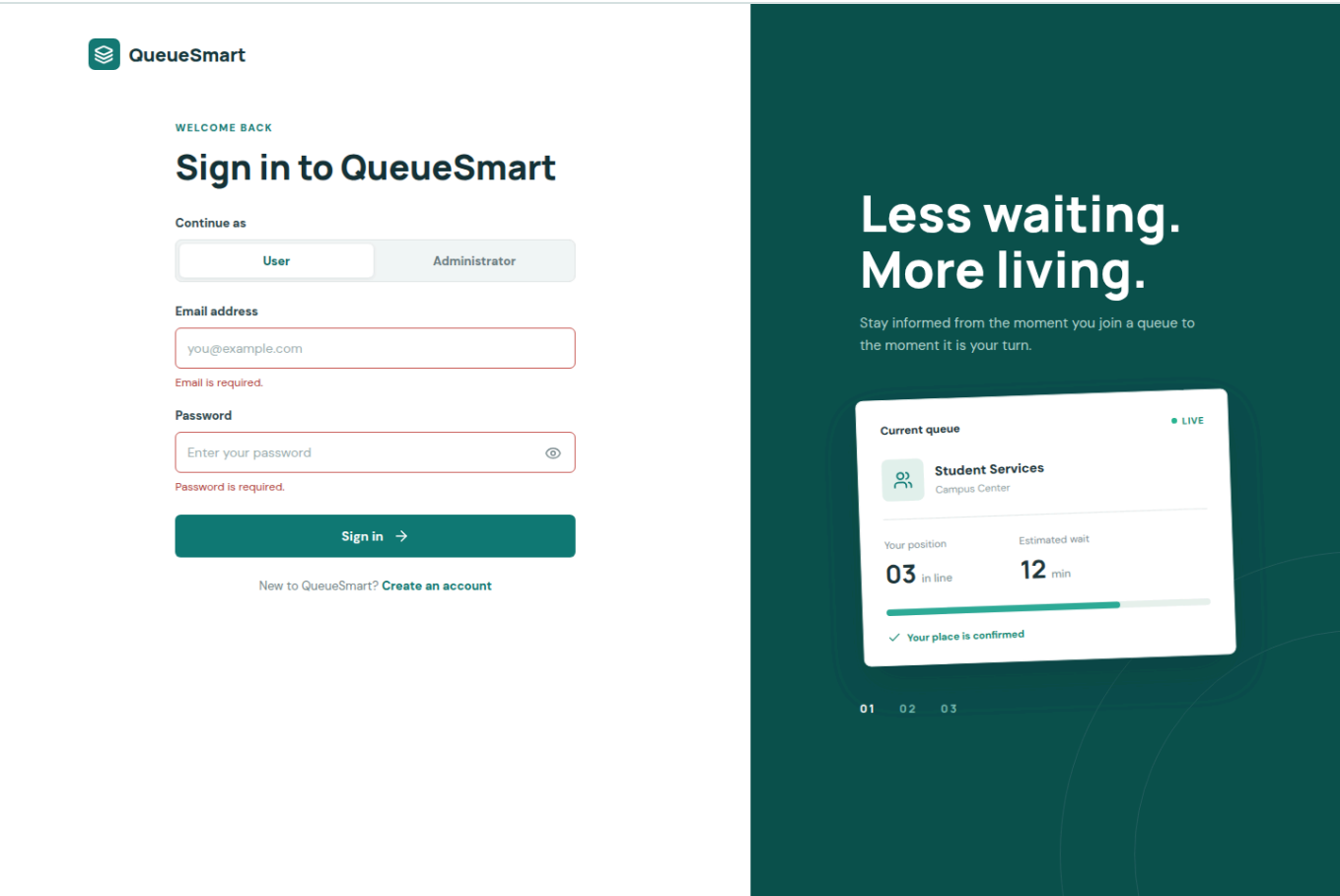


Figure 1b. Login validation. Required email and password, email format, and an 8–128 character password rule.

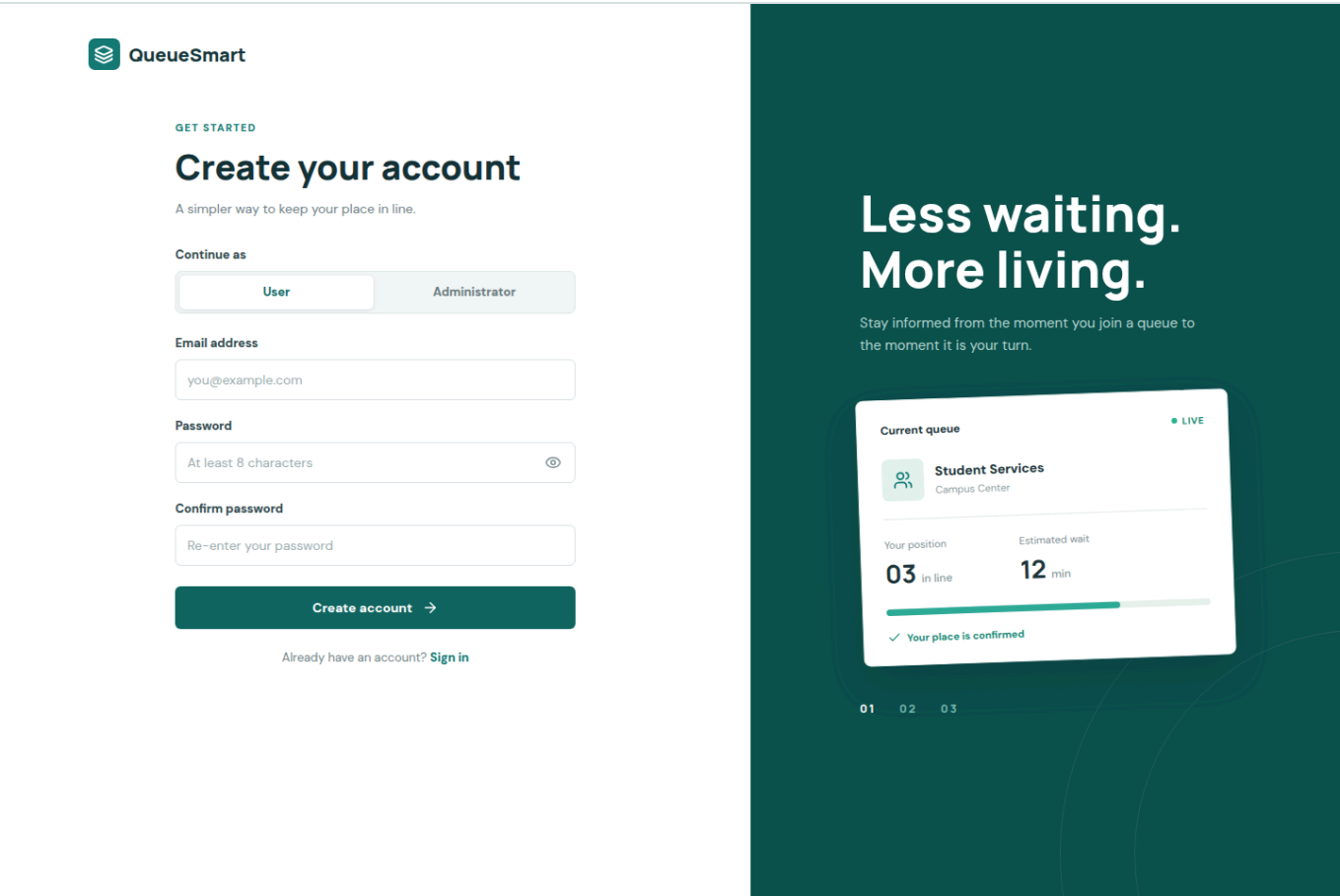


Figure 2. Registration. Email, password, confirm password, and role selection. New accounts can continue as a user or an administrator.

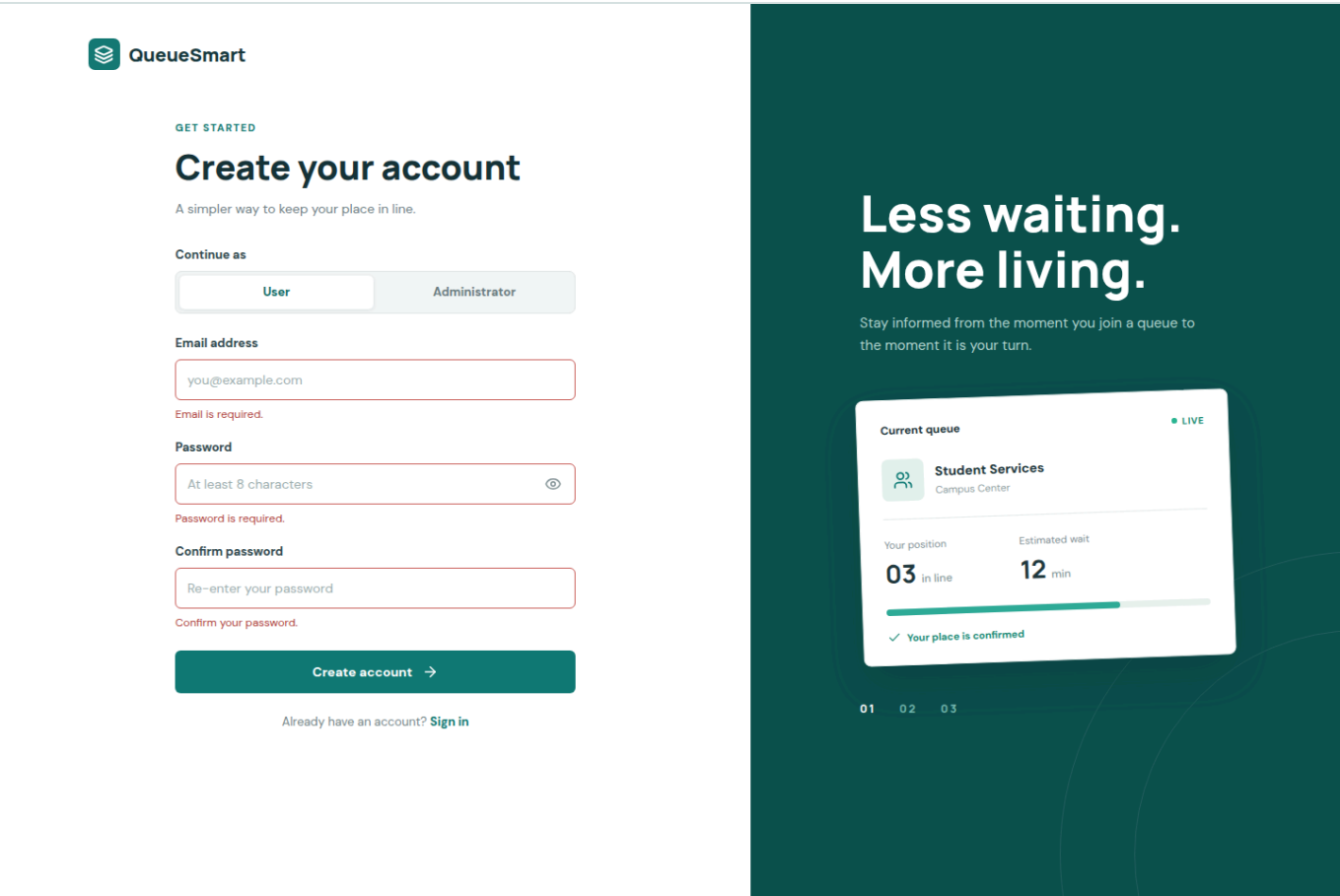


Figure 2b. Registration validation. Required fields, email type, password length, and a matching confirm-password check.

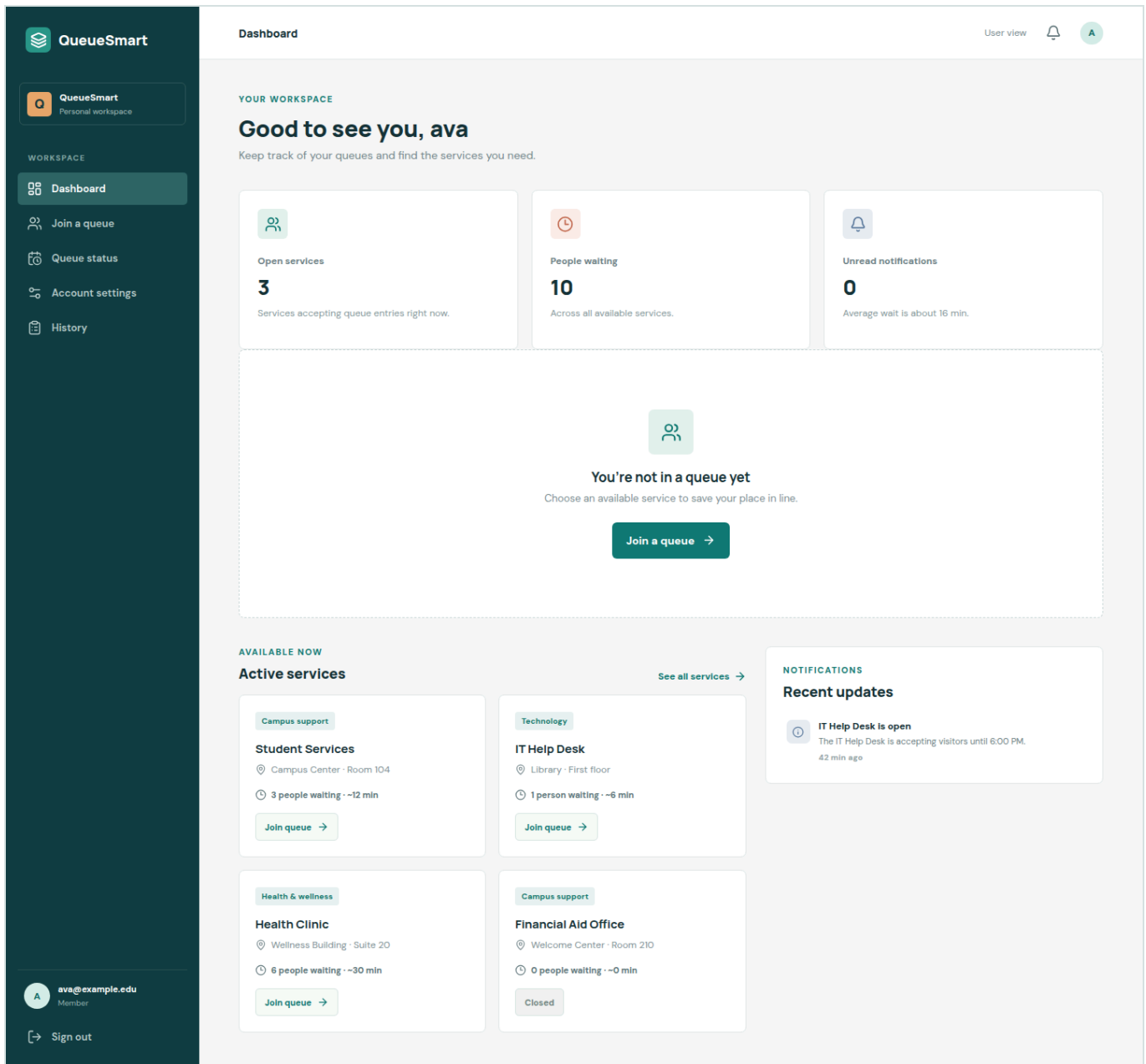


Figure 3. User Dashboard. Overview of open services and people waiting, current queue (or an empty state), popular / active services, and a notifications summary.

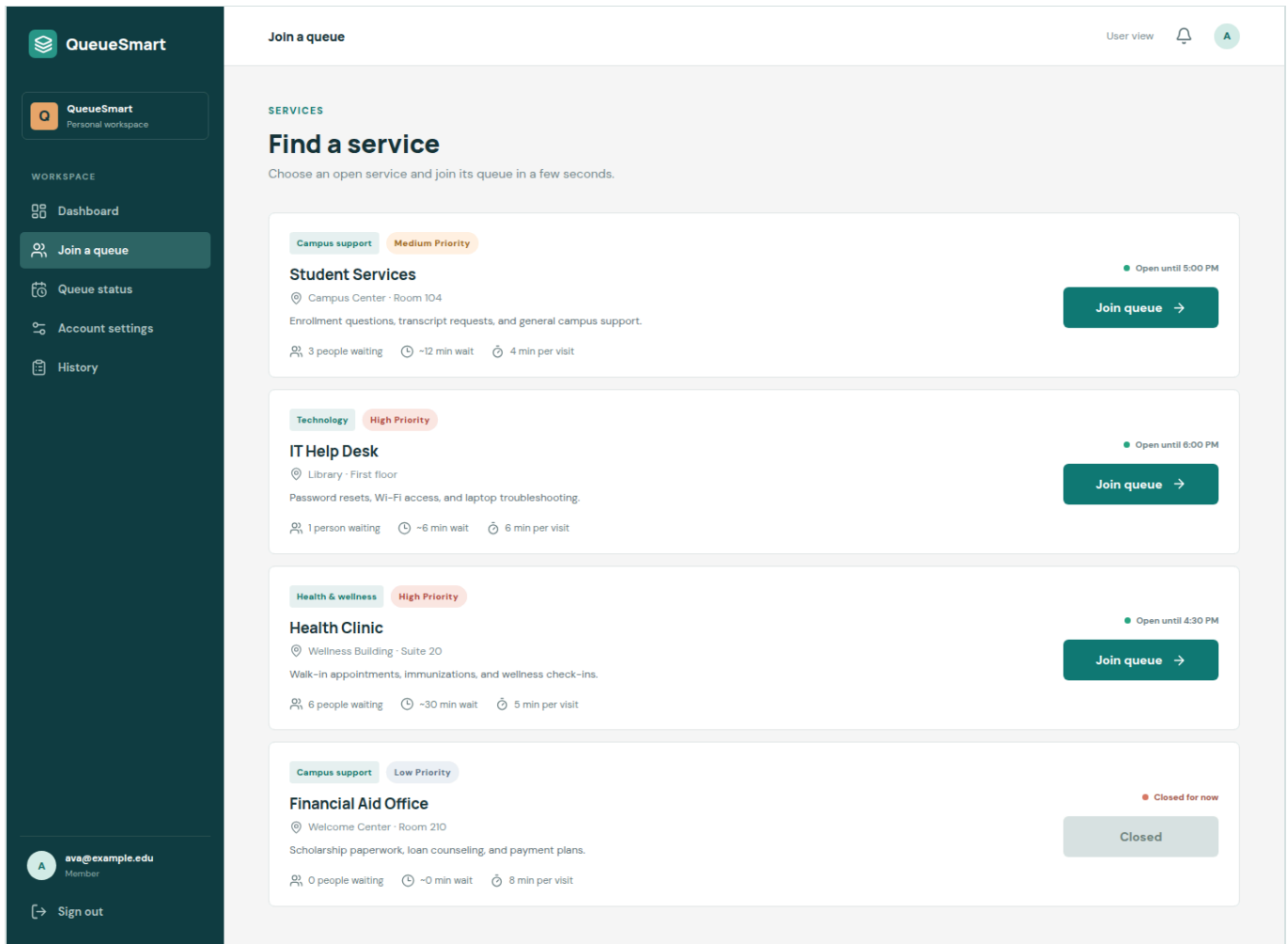


Figure 4. Join Queue. Select a service, see estimated wait time, and join or leave. Closed services cannot be joined. Only one active queue is allowed at a time.

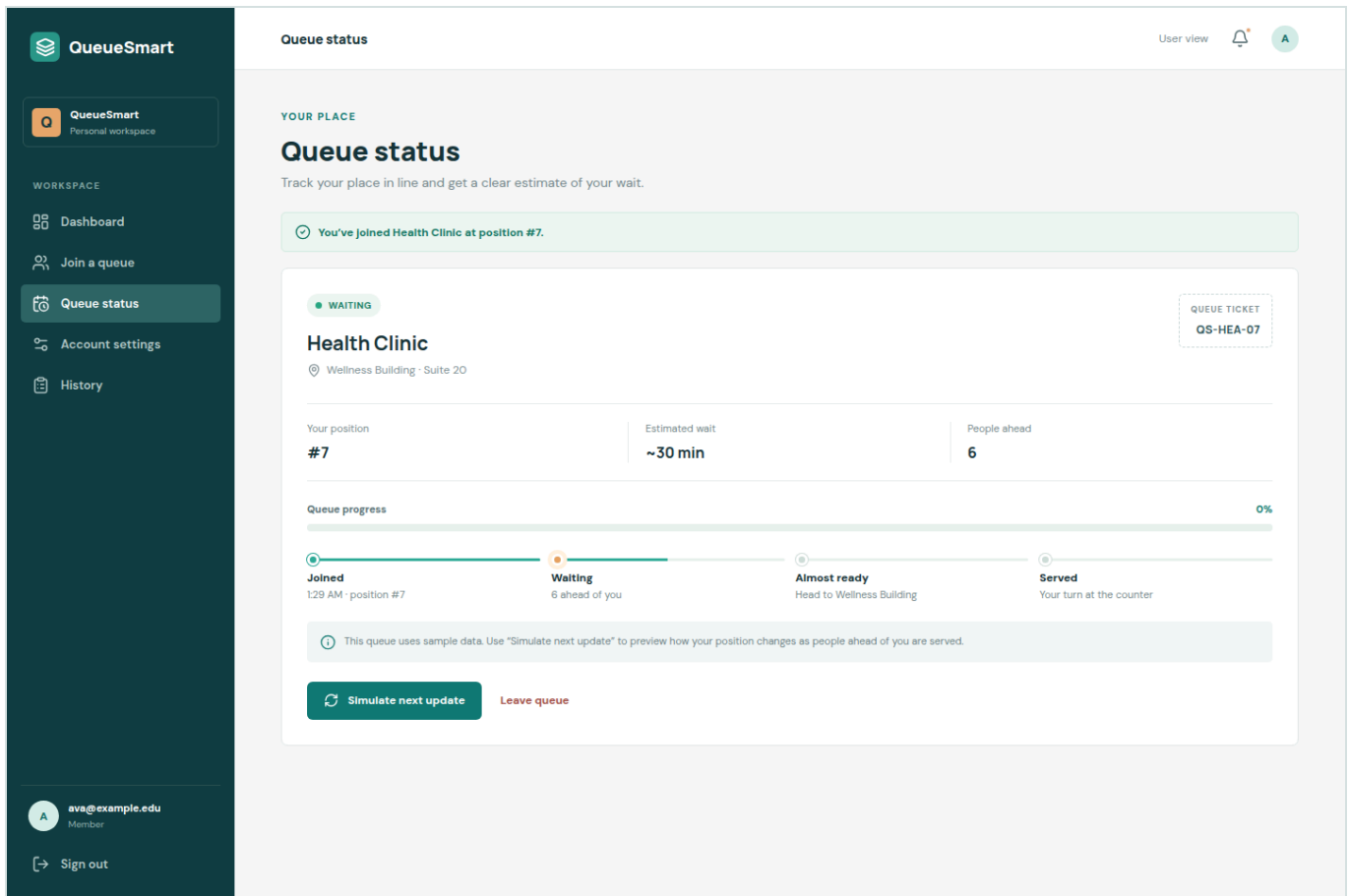


Figure 5. Queue Status. Current position, estimated wait, people ahead, ticket number, and status updates (Waiting, Almost Ready, Served). "Simulate next update" previews movement through the mock line.

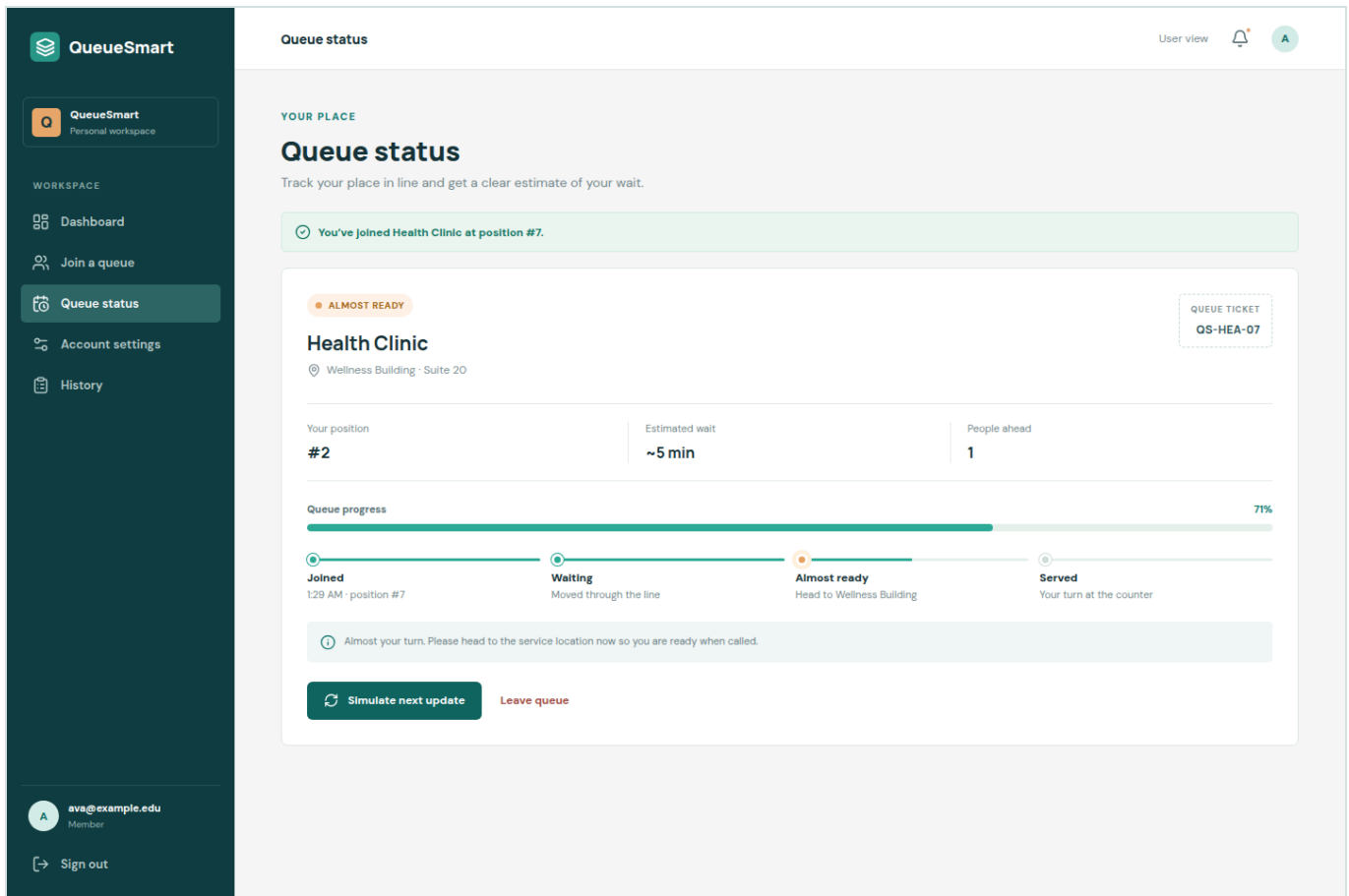


Figure 5b. Queue Status — Almost Ready. Status badge, progress, and timeline when the visitor is near the front of the line.

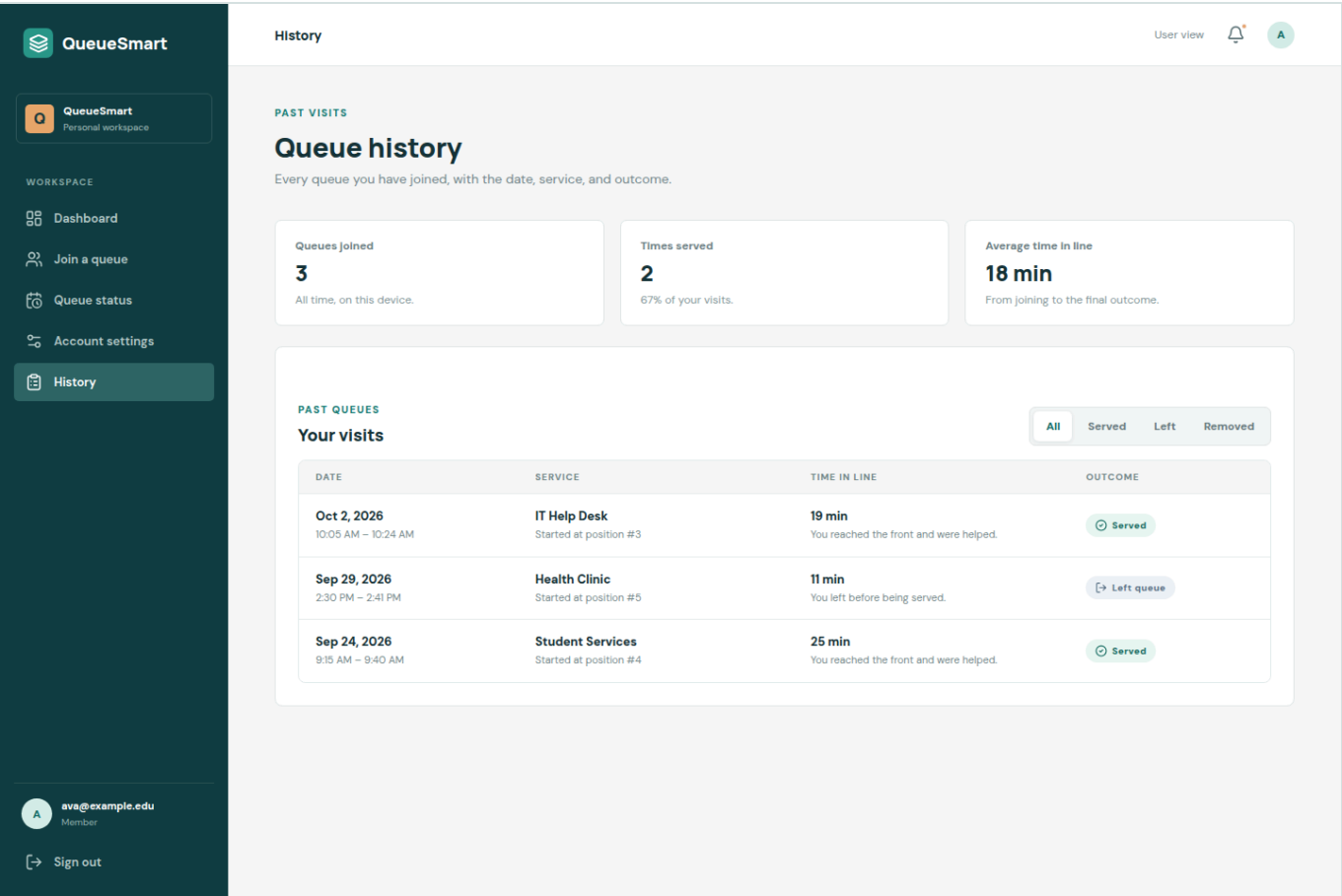


Figure 6. History. Past queues with date, service name, time in line, and outcome (Served, Left queue, Removed). Filter chips and summary stats are included.

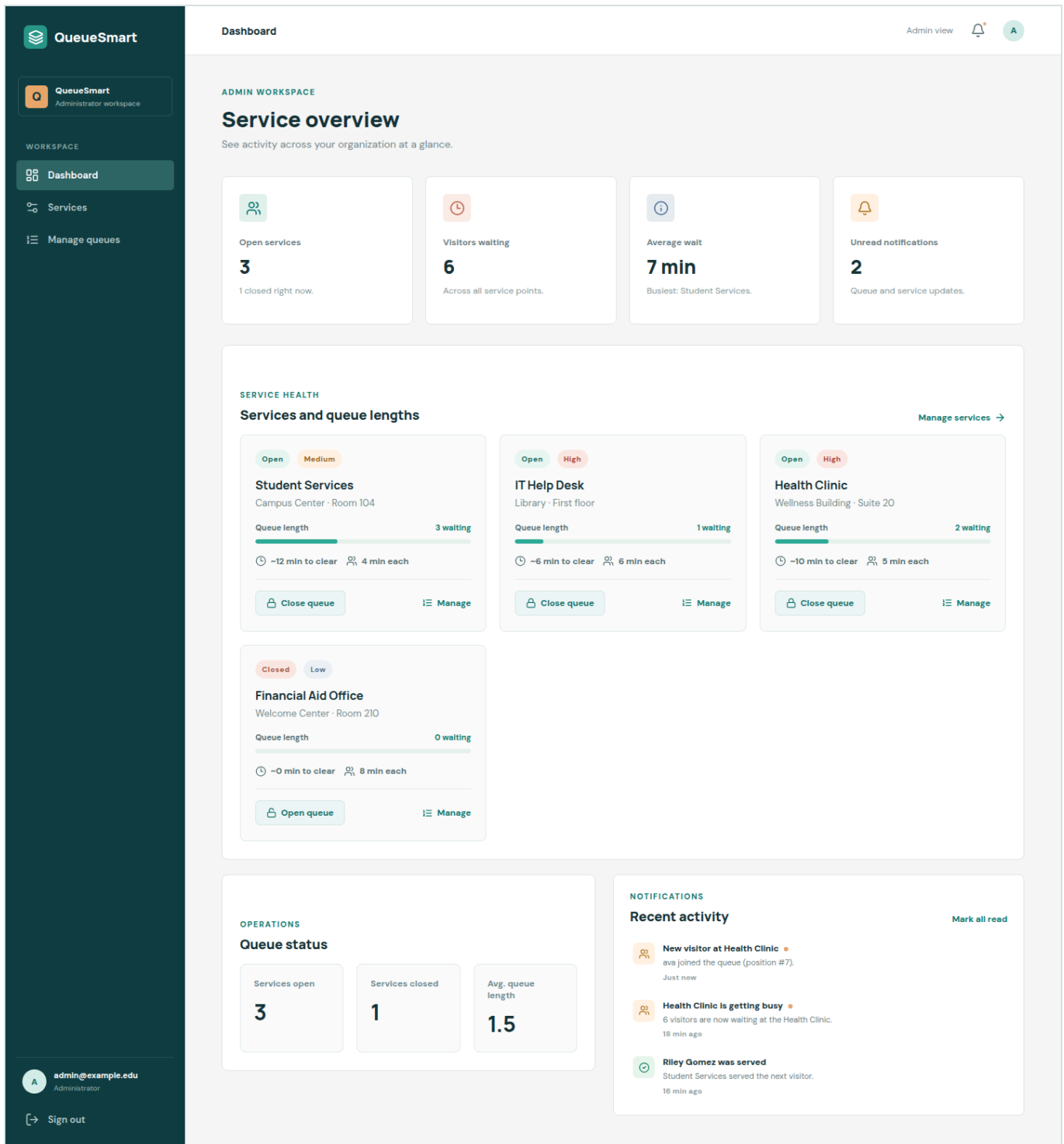


Figure 7. Admin Dashboard. List of services, current queue lengths, and quick actions to open or close a queue. Recent activity notifications appear on the same page.

 QueueSmart

QueueSmart

Administrator workspace

WORKSPACE

Dashboard

Services

Manage queues

admin@example.edu

Administrator

[-> Sign out

Services

Admin view



A

ADMIN WORKSPACE

Services

Create and manage the services your organization offers.

MANAGE

Service management

+ Create service

CATALOG

Current services

4 services

Open

Medium Priority

Student Services

Campus support · Campus Center · Room 104 · until 5:00 PM

Enrollment questions, transcript requests, and general campus support.

 Queue  Edit

Close queue

Delete

 3 waiting  4 min per visit

Open

High Priority

IT Help Desk

Technology · Library · First floor · until 6:00 PM

Password resets, Wi-Fi access, and laptop troubleshooting.

 Queue  Edit

Close queue

Delete

 1 waiting  6 min per visit

Open

High Priority

Health Clinic

Health & wellness · Wellness Building · Suite 20 · until 4:30 PM

Walk-in appointments, immunizations, and wellness check-ins.

 Queue  Edit

Close queue

Delete

 2 waiting  5 min per visit

Closed

Low Priority

Financial Aid Office

Campus support · Welcome Center · Room 210 · until 4:00 PM

Scholarship paperwork, loan counseling, and payment plans.

 Queue  Edit

Open queue

Delete

 0 waiting  8 min per visit

Figure 8. Service Management. Admins can create, edit, open/close, or delete services. Priority and expected duration are shown on each row.

Delete

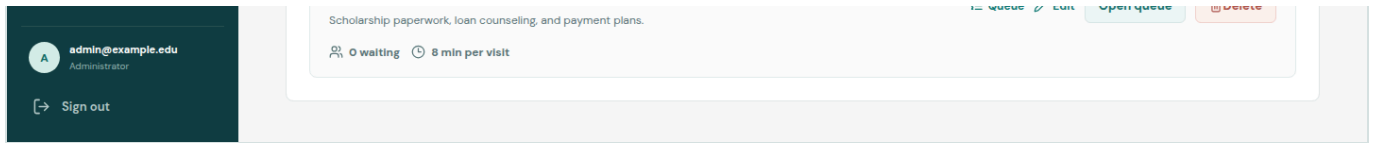


Figure 8b. Create / Edit Service form. Service Name (required, max 100 characters), Description (required), Expected Duration in minutes (required), Priority (low / medium / high), category, location, and closing time.

 QueueSmart

QueueSmart

Administrator workspace

WORKSPACE

Dashboard

Services

Manage queues

Services

Admin view





ADMIN WORKSPACE

Services

Create and manage the services your organization offers.

MANAGE

Service management

+ Create service

NEW

Create service

Service name *

e.g. Registrar's Office

Service name is required.

Description *

What does this service help visitors with?

Description is required.

Expected duration (minutes) *

10

Average time spent serving one visitor.

Priority level *

Medium

High-priority services are highlighted for members.

Category *

e.g. Campus support

Category is required.

Open until *

05:00 PM

Location *

Building - Room

Location is required.

☒ Queue is open and accepting visitors

Cancel

Create service →

CATALOG

Current services

4 services

Open

Medium Priority

Student Services

Campus support · Campus Center · Room 104 · until 5:00 PM

Enrollment questions, transcript requests, and general campus support.

 Queue  Edit

Close queue

Delete

 3 waiting  4 min per visit

Open

High Priority

IT Help Desk

Technology · Library · First floor · until 6:00 PM

Password resets, Wi-Fi access, and laptop troubleshooting.

 Queue  Edit

Close queue

Delete

 1 waiting  6 min per visit

Open

High Priority

Health Clinic

Health & wellness · Wellness Building · Suite 20 · until 4:30 PM

Walk-in appointments, immunizations, and wellness check-ins.

 Queue  Edit

Close queue

Delete

 6 waiting  5 min per visit

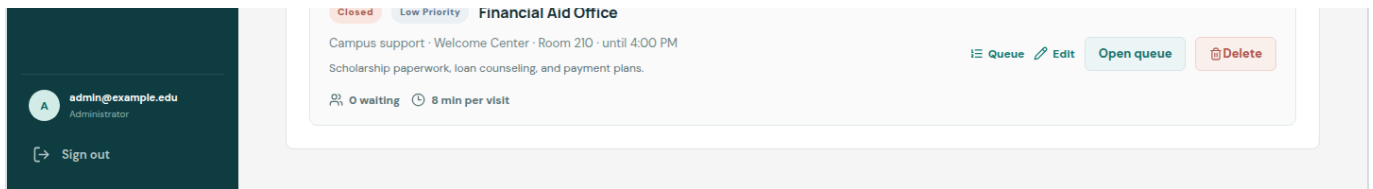


Figure 8c. Service form validation. Required fields, length limits, numeric duration, and a time input for “Open until.”

ADMIN WORKSPACE

Queue management

Manage visitors and keep each service moving.

Student Services 3

IT Help Desk 1

Health Clinic 6

Financial Aid Office 0

 10 waiting across 4 services

Open

Medium Priority

Student Services

 Close queue Serve next Campus Center - Room 104 · Open until 5:00 PM

Waiting

3

Expected duration

4 min

Total wait to clear

~12 min

Served recently

1

1

Avery Chen **NEXT UP**

avery.chen@example.edu

 Joined 1:16 AM (14 min ago) · est. 0 min wait Remove

2

Jordan Patel

jordan.patel@example.edu

 Joined 1:21 AM (9 min ago) · est. 4 min wait Remove

3

Sam Okafor

sam.okafor@example.edu

 Joined 1:27 AM (3 min ago) · est. 8 min wait Remove

RECENTLY SERVED

Completed visits



Riley Gomez

Served at 1:14 AM

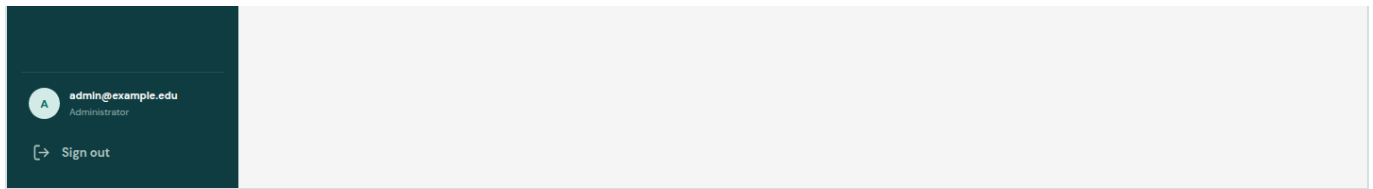


Figure 9. Queue Management. View the queue for a selected service. Reorder visitors, remove a visitor (UI only, with confirmation), and serve the next user. Recently served visits are listed under the line.

QueueSmart

QueueSmart

Administrator workspace

WORKSPACE

Dashboard

Services

Manage queues

Manage queues

Admin view

A

ADMIN WORKSPACE

Queue management

Manage visitors and keep each service moving.

Student Services3

IT Help Desk1

Health Clinic6

Financial Aid Office0

10 waiting across 4 services

Open

High Priority

Health Clinic

Close queue

Serve next

Wellness Building · Suite 20 · Open until 4:30 PM

Waiting6

Expected duration5 min

Total wait to clear~30 min

Served recently0

1

Taylor Brooks

NEXT UP

taylor.brooks@example.edu

Joined 12:59 AM (31 min ago) · est. 0 min wait

Remove

2

Casey Nguyen

casey.nguyen@example.edu

Joined 1:06 AM (24 min ago) · est. 5 min wait

Remove

3

Jamie Rivera

jamie.rivera@example.edu

Joined 1:11 AM (19 min ago) · est. 10 min wait

Remove

4

Drew Kim

drew.kim@example.edu

Joined 1:18 AM (12 min ago) · est. 15 min wait

Remove

5

Quinn Alvarez

quinn.alvarez@example.edu

Joined 1:22 AM (8 min ago) · est. 20 min wait

Remove

6

Reese Thompson

reese.thompson@example.edu

Joined 1:28 AM (2 min ago) · est. 25 min wait

Remove

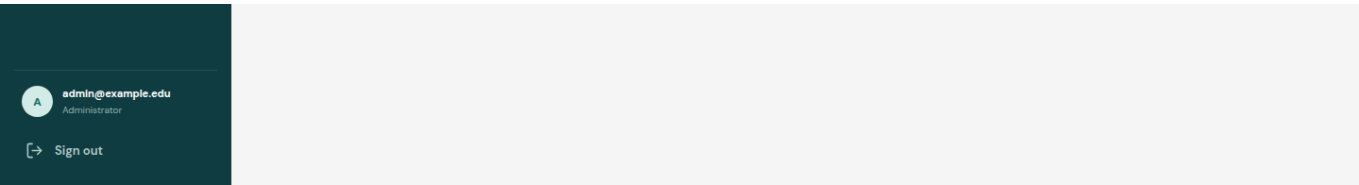


Figure 9b. Queue Management — selected service. Health Clinic line with live counts, move up / down, remove, and serve next.

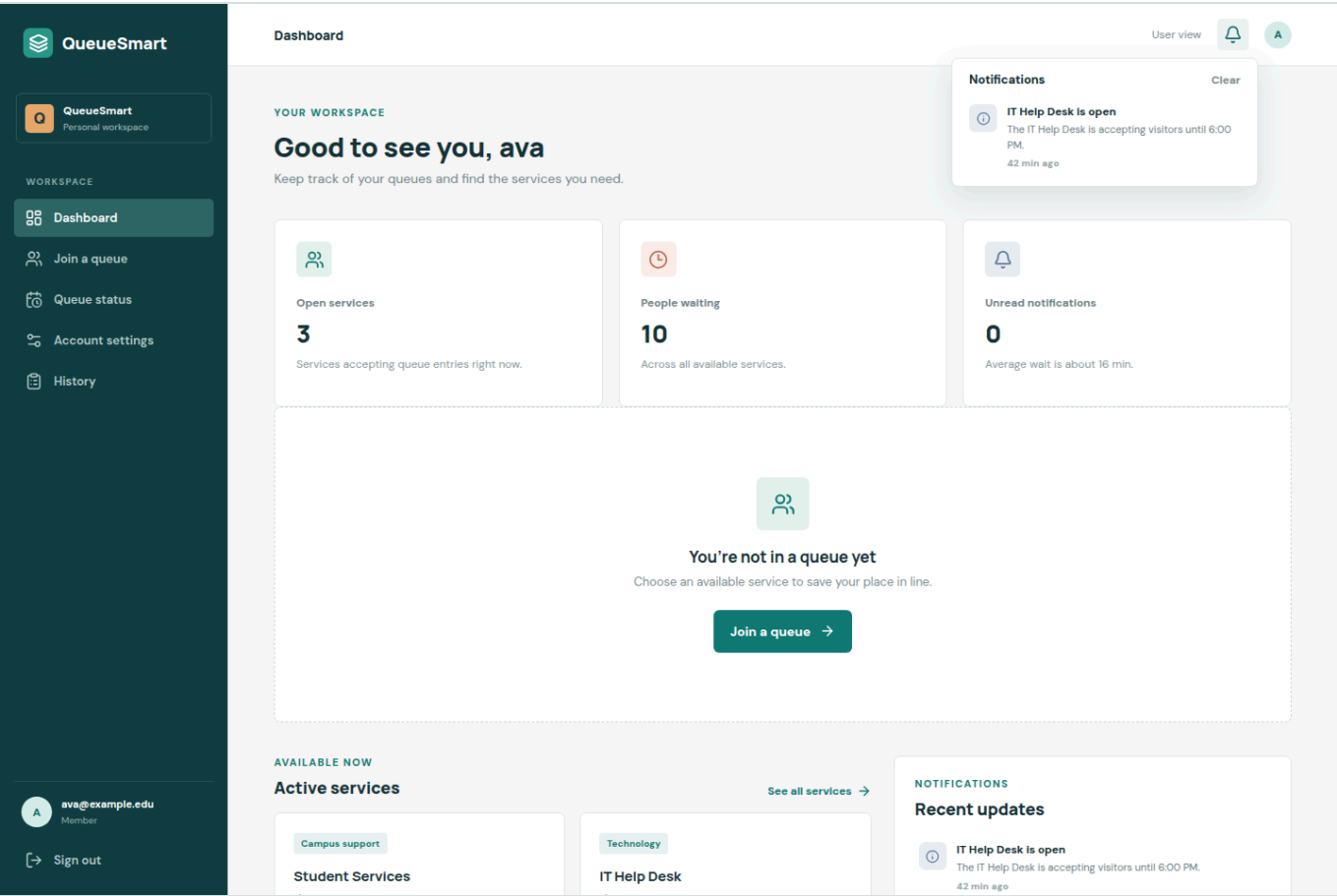


Figure 10. Notifications (in-app). Queue updates and status changes appear in the top-bar notification center (unread badge, mark read / mark all read / clear). The user dashboard also shows a notifications summary.

5. Team Contribution Table

TAs can confirm the rows below against GitHub commit history on github.com/Parham-Ebrahimi/QueueSmart.

Group member name	What is your contribution?	Discussion notes
Parham Ebrahimi	Authentication screens (login and registration), email/password validation, shared navigation and sidebar, base layout and CSS, shared Brand and FormField components.	Owned the UI foundation so later screens could reuse one visual system. Established the User vs Administrator workspace split.
Mustafa Ahmed	User Dashboard, Join Queue, Queue Status, mock queue join/leave/simulate-next, account settings, and the first mock service data.	Implemented the member journey from “not in a queue” through waiting, almost ready, and served. Kept wait-time copy consistent with the A1 design.
Kevin Santhosh	Admin Dashboard, Service Management (create / edit / delete), open and close queue controls, and the first pass of service-form validation.	Built the administrator catalog so services exist for both dashboards. Open/close is available from the dashboard and the service list.
Andrew Andari	Queue Management (service selector, reorder, remove, serve-next simulation, recently served), History screen, in-app notifications (top bar + dashboard summary), service-form spec alignment (description, priority, 100-character name, expected duration), and the shared persisted mock-data layer. Labeled screenshots and this submission document.	Finished the remaining A2 screens and connected user and admin views so joining a queue, serving the next visitor, and history/notifications all use one mock source of truth. Commits authored as wynetye.

Appendix — Assignment scope checklist

Required item	Status
Login and registration, email as username, password, client-side validation	Complete
User Dashboard (queue overview, active services, notifications summary)	Complete
Join Queue (select service, estimated wait, join / leave)	Complete
Queue Status (position, wait, Waiting / Almost Ready / Served)	Complete
History (date, service, outcome)	Complete
Admin Dashboard (services, queue lengths, open/close)	Complete
Service Management (name, description, expected duration, priority)	Complete
Queue Management (view line, reorder / remove, serve next)	Complete
In-app notifications for queue updates and status changes	Complete
Client-side validations (required, length, email, numbers, time)	Complete
This PDF / Word document for Canvas (code stays on GitHub only)	This file

Backend functionality is intentionally not implemented. Mock data is stored in the browser and is acceptable for Assignment 2. The same UI is intended for reuse in Assignment 3 (API integration) and Assignment 4 (data design).